

Assignment 1 – Checkpoint

(I'm not so familiar with notebooks, so I hope this will do.)

1. The BPE vocabulary sizes you chose and why

I chose to use **10,000** for my small BPE tokenizer and **20,000** for the big one, since the alphabet alone (= all characters in the training set + special tokens) comprised roughly 9,450 tokens. A 2,000-vocabulary BPE tokenizer would therefore not include all characters in the training set, let alone any useful merged tokens. This would kind of defeat the point of using a BPE tokenizer. With 10,000, at least there is some room for merged tokens (about 550), and with 20,000 there is a lot of room, so I find this comparison more meaningful than the suggested 2,000 vs. 10,000.

2. Your initial hypotheses about how the three tokenizers will behave

(I'm not really sure what "behave" means here?)

I expect the character-level tokenizer to split up texts literally character by character, while the small BPE tokenizer will combine a few of the characters and the big BPE tokenizer will combine not only many more individual characters but also groups of characters. Based on this, I assume the character-level tokenizer will be the least efficient in terms of token usage, with the small BPE tokenizer being slightly more efficient and the big tokenizer being much more efficient.

3. The parameter counts of your three models

Using PyTorch's `model.parameters()`, I get these parameter counts for my three models:

- Character-level tokenizer model: **6,490,340**
- Small BPE tokenizer model: **6,775,568**
- Big BPE tokenizer model: **11,905,568**

Using the formula given for the number of parameters only in the input embedding matrix (hidden dimensions x vocabulary size), I get these parameter counts:

- Character-level tokenizer model: $256 \times 9,444 = \mathbf{2,417,664}$
- Small BPE tokenizer model: $256 \times 10,000 = \mathbf{2,560,000}$
- Big BPE tokenizer model: $256 \times 20,000 = \mathbf{5,120,000}$

4. One English, one Turkish, and one Chinese sentence tokenized by all three tokenizers

(Note that the underscores in the BPE tokenizations below is due to the Metaspace pre-tokenization I am using. This preserves separation between words while simultaneously preventing tokenization over word boundaries, which I think makes more sense.)

Sample sentence (EN): In 1536, the Act of Union was passed under Henry's rule which had a long-lasting effect on Wales as a nation.

- **Character-level:** In 1536, the Act of Union was passed under Henry's rule which had a long-lasting effect on Wales as a nation.
- **Small BPE:** _In _1536, _the _Act _of _Union _was _passed _under _Henry's _rule _which _had _a _long - l ast ing _e f f e c t _on _W al es _as _a _n ation .
- **Big BPE:** _In _1536, _the _Act _of _Union _was _pass ed _under _Henry's _rule _which _had _a _long - l ast ing _e f f e c t _on _W al es _as _a _n ation .

Sample sentence (TR): Kayseri iline 54 km, Bünyan ilçesine 19 km uzaklıktadır.

- **Character-level:** Kayseri iline 54 km, Bünyan ilçesine 19 km uzaklıktadır.
- **Small BPE:** _K a y s e r i _ i l i n e _ 5 4 _ k m , _ B ü n y a n _ i l ç e s i n e _ 1 9 _ k m _ u z a k l ı k t a d ı r .
- **Big BPE:** _K a y s e r i _ i l i n e _ 5 4 _ k m , _ B ü n y a n _ i l ç e s i n e _ 1 9 _ k m _ u z a k l ı k t a d ı r .

Sample sentence (ZH): 藝術家會在作品中控制灰分的百分比，若降低灰的比例，控制的程度更高 Ford & Impey, 46-50。

- **Character-level:** 藝術家會在作品中控制灰分的百分比，若降低灰的比例，控制的程度更高 Ford & Impey, 46-50。
- **Small BPE:** _ 藝 術 家 會 在 作 品 中 控 制 灰 分 的 百 分 比 ， 若 降 低 灰 的 比 例 ， 控 制 的 程 度 更 高 _ F o r d _ & _ I m p e y , _ 4 6 - 5 0 _ 。
- **Big BPE:** _ 藝 術 家 會 在 作 品 中 控 制 灰 分 的 百 分 比 ， 若 降 低 灰 的 比 例 ， 控 制 的 程 度 更 高 _ F o r d _ & _ I m p e y , _ 4 6 - 5 0 _ 。

5. Preliminary tokenizer statistics for the three languages

Vocabulary size:

- Character-level tokenizer: **9,444**
- Small BPE tokenizer: **10,000**
- Big BPE tokenizer: **20,000**

Average Unicode characters/token (on the validation data across all three languages, since some tokens are shared):

- Character-level tokenizer: **1.0008** (since I add [UNK] and [EOS])
- Small BPE tokenizer: **1.1036**
- Big BPE tokenizer: **2.5255**

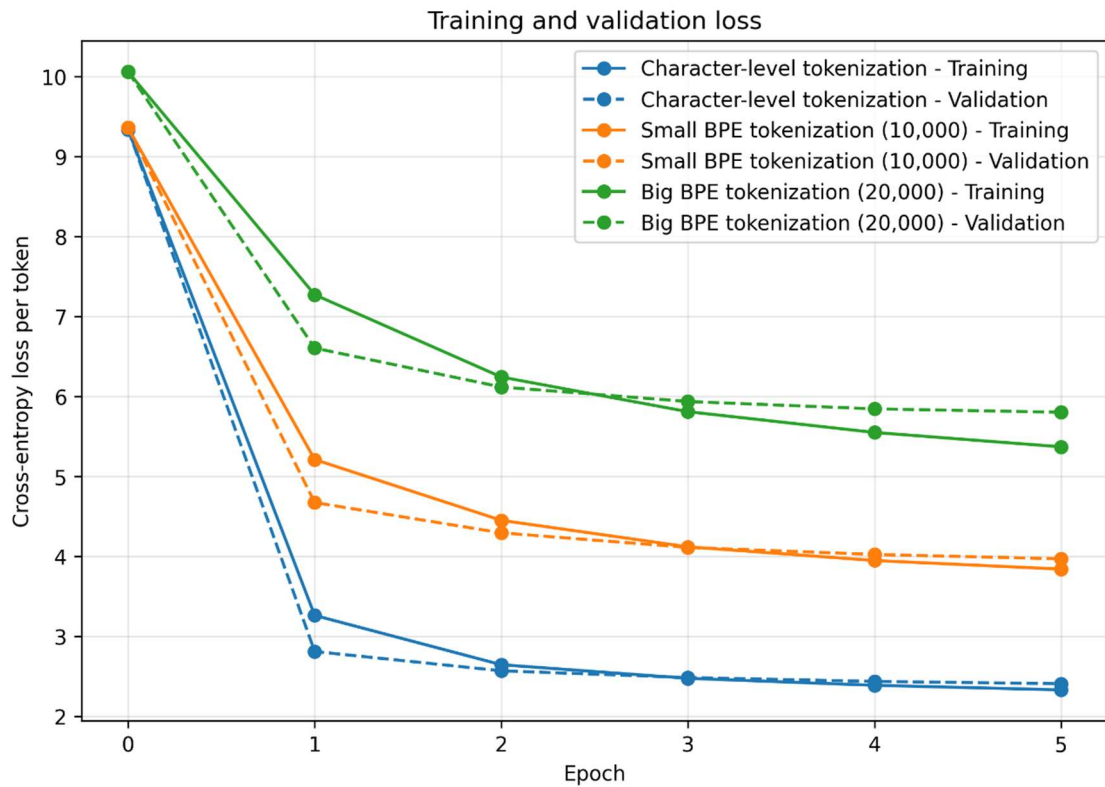
Total tokens (on validation data):

	en	tr	zh
Char-level	368,269	368,383	368,266
Small BPE	174,617	182,138	346,050
Big BPE	107,828	106,077	267,425

Average tokens per sentence (on validation date):

	en	tr	zh
Char-level	81.1	98.6	38.5
Small BPE	38.5	49.0	36.2
Big BPE	23.8	28.4	28.0

6. At least one preliminary training or validation curve



7. One result or observation that you find interesting so far

I'm surprised that the small BPE tokenizer is so competitive for English and Turkish despite having so little room for merged tokens. I guess this suggests that the first merges matter the most, which would make sense since those are also the most common multi-character combinations.

8. One question that you want your final experiments to help answer

Is there any point in using a character-level tokenizer if you have a BPE tokenizer readily available? (Basically, what is the benefit of using a character-level tokenizer instead, if any?)